

WZUM Projekt

Bartłomiej Biesiada

Czerwiec 2025

1 Opis rozwiązania

1.1 Przygotowanie danych

Proces przygotowania danych rozpoczął się od pobrania podstawowych informacji za pośrednictwem oficjalnego API `nba.com`. Ze względu na ograniczony zakres danych udostępnianych przez to źródło, zdecydowano się na rozszerzenie zbioru poprzez zastosowanie techniki web scrapingu, wykorzystując w tym celu stronę `basketball-reference.com`. Operacja ta została zrealizowana za pomocą autorskiego skryptu `web_scrap.py`.

Po zebraniu wszystkich niezbędnych informacji dane zostały odpowiednio oczyszczone, połączone i ujednolicone, a następnie wzbogacone o dodatkowe atrybuty (skrypt `pre_proccesing.py`). Na podstawie plików z wynikami konkursów *All-NBA* oraz *All-Rookie* przypisano zawodnikom informacje liczbowe o zdobytych nagrodach w konkretnych sezonach (np. pierwsza piątka oznaczona jest jako 1).

Kolejnym etapem była analiza korelacji poszczególnych zmiennych z docelową klasą, czyli obecnością nagrody. Pozwoliło to na wybór najbardziej znaczących statystyk, z uwzględnieniem ich wzajemnych zależności oraz wpływu na wynik modelu. Korelacje przedstawiono na rysunkach 1 oraz 2. Wybór statystyk polegał na ich ręcznym doborze z uwzględnieniem niskiej wzajemnej korelacji oraz wysokiej korelacji z klasą wyjściową. Na wspomnianych rysunkach zmienne zostały uporządkowane względem bezwzględnej wartości korelacji z klasą.

W przypadku danych dotyczących zawodników debiutujących w lidze uwzględniono wyłącznie tych graczy, którzy w danym sezonie formalnie posiadali status *rookies*, co zapewniło spójność i adekwatność analizowanego zbioru danych.

Warto również zaznaczyć, że każdorazowa zmiana w zestawie danych była ewaluowana za pomocą skryptu zliczającego punkty, udostępnionego przez prowadzącego.

Końcowy zbiór danych zawierał następujące statystyki:

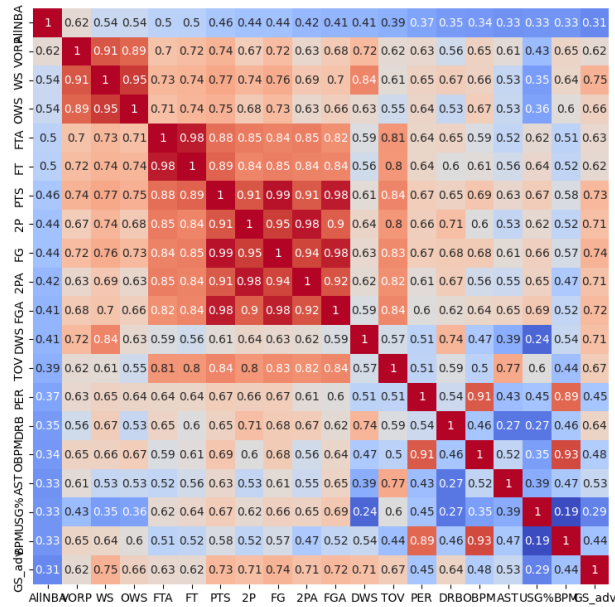
- **Wspólne:**
 - Z pliku `per_game`: PTS, DRB, TOV
- **Dla klasyfikacji *All-NBA*:**

- Z pliku advanced: VORP, DWS, PER, G
- Z pliku per_game: FTA, PTS

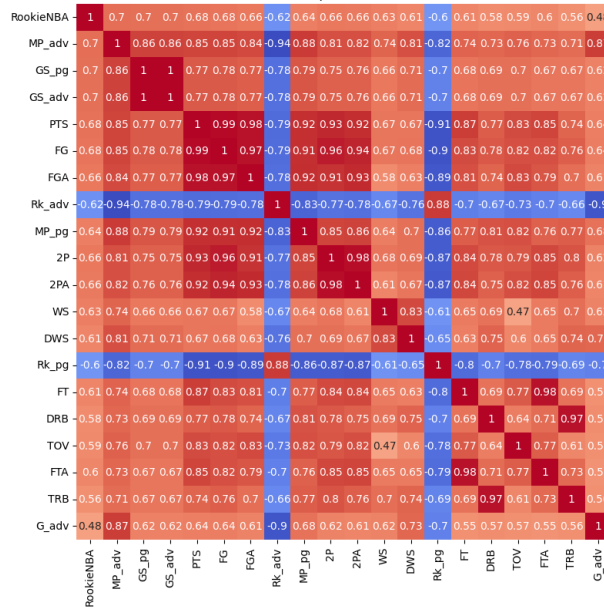
• Dla klasyfikacji *Rookie*:

- Z pliku advanced: GS, MP, WS
- Z pliku per_game: 2P, FT

Dodatkowo, z końcowego zbioru usunięto zawodników, którzy rozegrali mniej niż 64 mecze w sezonie, aby wyeliminować przypadki nieklasyfikujące się do nagrody.



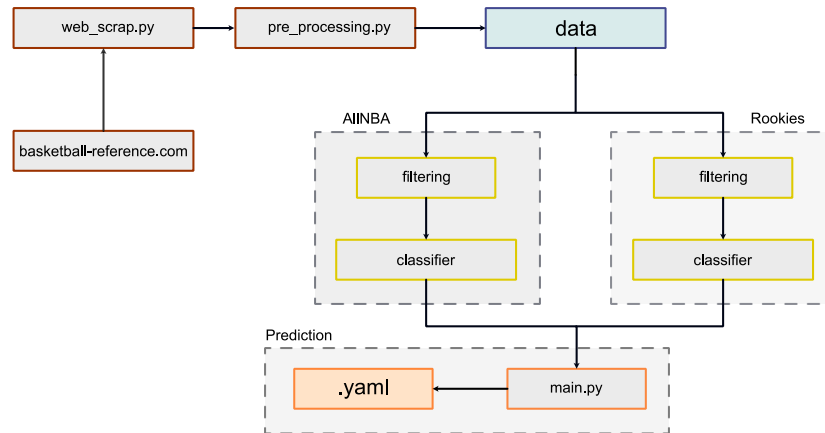
Rysunek 1: Korelacja między 20 najbardziej znaczącymi danymi



Rysunek 2: Korelacja między 20 najbardziej znaczącymi danymi dla Rookies

1.2 Schemat klasyfikatora

Schemat działania klasyfikatora przedstawiono na rysunku 3. Dane wejściowe są najpierw przetwarzane przez dwa klasyfikatory filtrujące, których zadaniem jest wybranie najlepszych 30 zawodników z danego sezonu. Ma to na celu zredukowanie liczby zawodników bez nagród, których znaczna przewaga liczebna mogłaby negatywnie wpływać na efektywność końcowego modelu.



Rysunek 3: Schemat działania klasyfikatora warstwowego

Dopiero tak przefiltrowany zbiór danych trafia do właściwego klasyfikatora, który dokonuje końcowej predykcji. Klasyfikatory zostały wybrane ze zbioru modeli dostępnych w bibliotekach `scikit-learn`, `catboost` oraz `xgboost`. Ostateczny wybór klasyfikatorów odbywał się na podstawie jakości predykcji ocenianej za pomocą skryptu zliczającego punkty, udostępnionego przez prowadzącego.

Modele filtrujące:

```
"Logistic Regression",
"Random Forest",
"Gradient Boosting",
"AdaBoost",
"SVC",
"GaussianNB",
"k-Nearest Neighbors",
"CatBoost",
"XGBoost"
```

Modele właściwe (klasyfikatory końcowe):

```
"Logistic Regression",
"Random Forest",
"Gradient Boosting",
"AdaBoost",
"SVC",
"GaussianNB",
"k-Nearest Neighbors",
"CatBoost",
```

Najlepszy wynik: Dla końcowego zbioru danych najwyższy wynik uzyskano przy zastosowaniu klasyfikatora `GaussianNB` jako filtra oraz `Random Forest` jako modelu końcowego. Zestaw ten osiągnął wynik **554 punktów**, co było najlepszym rezultatem spośród wszystkich przetestowanych konfiguracji.

Przykładowe wyniki jednej z prób klasyfikacji:

Kombinacja modeli (filtrujący + końcowy)	Wynik (Score)
k-Nearest Neighbors + XGBoost	506
AdaBoost + CatBoost	476
Logistic Regression + XGBoost	476
k-Nearest Neighbors + Random Forest	476
Logistic Regression + k-Nearest Neighbors	462
SVC + k-Nearest Neighbors	462
Gradient Boosting + k-Nearest Neighbors	462
Random Forest + Random Forest	462
CatBoost + Random Forest	462
GaussianNB + XGBoost	502
AdaBoost + SVC	500

1.3 Nieudane podejścia i wyciągnięte wnioski

W ramach projektu przetestowano również kilka alternatywnych podejść do problemu klasyfikacji, które ostatecznie okazały się mniej skuteczne niż finalna wersja (branch `two_clas`). Celem było sprawdzenie różnych metod podziału danych i klasyfikacji, co miało pomóc w lepszym odwzorowaniu struktury nagród w NBA.

Pierwsze podejście, dostępne na branchu `main`, zakładało istnienie jednego klasyfikatora, który przypisywał graczy do jednej z sześciu klas:

- 0 – brak nagrody,
- 1, 2, 3 – odpowiednio pierwsza, druga i trzecia drużyna All-NBA,
- 4, 5 – odpowiednio pierwsza i druga drużyna All-Rookie.

Nie dokonano tu rozdzielenia danych na dwie grupy (`rookie` i `all-nba`), co spowodowało mniejsze dopasowanie modelu do specyfiki nagród. Najlepszy uzyskany wynik z tego podejścia wyniósł 366 punktów.

Kolejną wersję zaimplementowano na branchu `one_class`. Tutaj zastosowano uproszczoną klasyfikację binarną (nagroda vs brak nagrody, klasy 0 i 1). Dalsze przypisanie gracza do odpowiedniej piątki (np. pierwsza drużyna All-NBA) odbywało się na podstawie poziomu pewności klasyfikatora co do przewidywanej klasy pozytywnej. To podejście pozwoliło uzyskać nieco lepszy wynik — 402 punkty — jednak nadal było zauważalnie gorsze od wersji finalnej.

Warto również zaznaczyć, że w obu przypadkach (zarówno na branchu `main`, jak i `one_class`) przeprowadzono procedurę strojenia hiperparametrów z wykorzystaniem metody `GridSearchCV`. Mimo przeszukania szerokiego zakresu parametrów nie zaobserwowano istotnej poprawy wyników klasyfikacji. W związku z tym, w dalszych etapach prac, zrezygnowano z tej metody na rzecz prostszych ustawień domyślnych lub ręcznie dobranych parametrów.

Oba podejścia dostarczyły cennych informacji o strukturze problemu i były pomocne w opracowaniu końcowej wersji klasyfikatora, w której zastosowano dwa oddzielne modele filtrujące oraz właściwą klasyfikację opartą na odrębnych zestawach cech i klas.